Modern PHP Cheatsheet (7.0 → 8.5)

The main syntax additions since PHP 7, grouped by theme. The version in brackets is when the feature appeared: code using it won't run on an older PHP.

Types

```php
function f(int $a, string $b): bool { }          // scalar types + return type   [7.0]
declare(strict_types=1);                          // no silent conversion         [7.0]

function find(?int $id): ?User { }                // nullable                     [7.1]
function log(string $m): void { }                 // no return value              [7.1]
function walk(iterable $items) { }                // array or Traversable         [7.1]
function save(object $o) { }                      // any object                   [7.2]

class User { public int $id; private ?string $email = null; }  // typed properties [7.4]

function parse(int|string $v): int|float { }      // union types                  [8.0]
function dump(mixed $v) { }                       // anything                     [8.0]
public function with(): static { }                // returns the calling class    [8.0]

function stop(): never { exit; }                  // never returns                [8.1]
function f(Countable&Iterator $x) { }             // intersection types           [8.1]

function f(): false|null { }                      // null, false, true standalone  [8.2]
function f((A&B)|null $x) { }                     // DNF types (mix | and &)       [8.2]
```

> Deprecated in **[8.4]**: implicit nullable `function f(Foo $x = null)`. Write `?Foo $x = null`.

Operators and short syntax

```php
$name = $_GET['name'] ?? 'Guest';                 // null coalescing              [7.0]
$a <=> $b;                                        // spaceship: -1, 0, 1          [7.0]
$user['role'] ??= 'member';                       // assign if null/unset         [7.4]

[$a, $b] = [1, 2];                                // short destructuring          [7.1]
['id' => $id, 'name' => $n] = $row;               // destructuring with keys      [7.1]
[$a, $b] = [$b, $a];                              // swap

$all = [...$first, ...$second];                   // spread in arrays             [7.4]
$all = [...$withStringKeys, ...$other];           // spread with string keys      [8.1]

$price = 1_000_000;                               // numeric separator            [7.4]
$perms = 0o755;                                   // explicit octal               [8.1]
"hello"[-1];                                      // negative string offset: "o"  [7.1]

$country = $user?->getAddress()?->country;        // nullsafe operator            [8.0]
$v = $x ?? throw new Exception('missing');        // throw is an expression       [8.0]

$slug = $title |> trim(...) |> strtolower(...);   // pipe operator                [8.5]
```

Functions

```php
$double = fn($x) => $x * 2;                       // arrow function              [7.4]

greet(name: 'Alice', greeting: 'Hi');             // named arguments             [8.0]
function f($a, $b,) { }  f(1, 2,);                // trailing commas (calls 7.3, params 8.0)

$len = strlen(...);                               // first-class callable        [8.1]
$m = $obj->method(...);
```

Classes and objects

```php
class Foo {
    private const SECRET = 'x';                    // constant visibility          [7.1]
    final public const VERSION = 2;                // final constants              [8.1]
    const string NAME = 'foo';                     // typed constants              [8.3]
}

$logger = new class implements Logger { };         // anonymous class              [7.0]

class Point {
    public function __construct(                   // constructor promotion        [8.0]
        public int $x = 0,
        public int $y = 0,
    ) {}
}

class Order {
    public function __construct(
        public readonly int $id,                   // readonly property            [8.1]
        private Logger $logger = new NullLogger(),// "new" as default value        [8.1]
    ) {}
}

readonly class Money { /* all properties readonly */ }   // readonly class         [8.2]

$obj::class;                                       // ::class on an object         [8.0]
Foo::{$constName};                                 // dynamic constant fetch       [8.3]
new Foo()->run();                                  // no parentheses needed        [8.4]
$copy = clone($order, ['status' => 'paid']);       // clone with new values        [8.5]
```

Dynamic properties (`$obj->undeclared = 1`) are **deprecated** since **[8.2]**: declare your properties.

Property hooks and asymmetric visibility [8.4]

```php
class User {
    public string $email {
        set => strtolower($value);                 // runs on every write
    }
    public string $fullName {
        get => "$this->first $this->last";         // computed (virtual) property
    }
    public private(set) int $id;                   // readable everywhere, writable inside only
}
```

Enums [8.1]

```php
enum Status: string {                              // "backed" enum (string or int)
    case Draft = 'draft';
    case Published = 'published';

    public function label(): string {
        return match ($this) {
            Status::Draft => 'Draft',
            Status::Published => 'Online',
        };
    }
}

$s = Status::Published;
$s->value;                      // 'published'
Status::from('draft');          // Status::Draft (error if invalid)
Status::tryFrom('nope');        // null
Status::cases();                // all cases
function publish(Status $s) { }  // type-safe: no more magic strings
```

Control flow and errors

```php
$label = match ($code) {                        // strict (===), returns a value [8.0]
    200, 201 => 'OK',
    404      => 'Not found',
    default  => 'Error',
};

try { } catch (TypeError | ValueError $e) { }     // multi-catch                  [7.1]
try { } catch (Exception) { }                     // catch without variable       [8.0]
```

Since **[7.0]**, most fatal errors are thrown as `Error`. Catch both kinds with `Throwable`.

Attributes [8.0]

Native metadata, replacing docblock annotations (Symfony routes, Doctrine mappings…).

```php
#[Route('/blog', name: 'blog')]
public function index() { }

#[\Override]               // error if the parent has no such method       [8.3]
#[\Deprecated]             // emits a deprecation when used                [8.4]
#[\SensitiveParameter]     // hides the value in stack traces              [8.2]
#[\NoDiscard]              // warns if the return value is ignored         [8.5]
```

New functions worth knowing

Function	Version
`random_int()`, `random_bytes()`	7.0
`array_key_first()`, `array_key_last()`, `is_countable()`	7.3
`str_contains()`, `str_starts_with()`, `str_ends_with()`	8.0
`get_debug_type()`	8.0
`array_is_list()`	8.1
`json_validate()`, `mb_str_pad()`	8.3
`array_find()`, `array_find_key()`, `array_any()`, `array_all()`	8.4
`mb_trim()`, `mb_ucfirst()`	8.4
`array_first()`, `array_last()`	8.5

Behavior changes that can surprise

Change	Version
`0 == "foo"` is now `false` (was `true`)	8.0
`"abc" * 2` throws `TypeError` instead of warning + 0	8.0
Default PDO error mode is exceptions	8.0
Passing `null` to non-nullable internal function params is deprecated	8.1
Dynamic properties deprecated	8.2
Implicit nullable parameter types deprecated	8.4

Release timeline

Version	Released	Headline
7.0	2015	2× faster, scalar types, `??`, `<=>`
7.4	2019	typed properties, arrow functions
8.0	2020	JIT, named args, `match`, union types, attributes
8.1	2021	enums, readonly, fibers
8.2	2022	readonly classes, DNF types
8.3	2023	typed constants, `#[\Override]`
8.4	2024	property hooks, asymmetric visibility
8.5	2025	pipe operator, `clone` with